TraceMaker

A learning, GPU-assisted PCB autorouter and placer for KiCad

Design, method and benchmark results

Development report

11 October 2026 (development branch `m14-bga`, after release 0.8.0; earlier editions: 2 and 6 October 2026)

Abstract

TraceMaker reads KiCad 4–10 boards directly, routes them with an exact-geometry lattice router that learns from its own failures, and writes the tracks and vias back into the `.kicad_pcb` file without disturbing anything else. It also places components with an analytic placer checked by the router. Every result is judged by KiCad's own design-rule checker (`kicad-cli pcb drc`). On Freerouting's PCBench benchmark set TraceMaker routes 100% of the sampled tier-A boards cleanly, 70.0% of tier B, 63.3% of tier C and 54.5% of tier D. Freerouting 2.5.0-RC12's published results on the same boards are 100%, 50.0%, 46.7% and 36.4%. No board in the final runs has a DRC error introduced by the router. On 60 boards never used during development, with Freerouting 2.5.0-RC12 and 1.9.0 run on the same machine and judged by the same KiCad DRC, TraceMaker's results are clean on 73–78% of boards against 3% and 32%, with 5% shorter tracks and 28% fewer bends than Freerouting 2.5, but more vias. This edition keeps all of that and adds Section 7, on the work of 7–11 October: routing of ball-grid-array (BGA) breakout boards and SoC fan-outs from tscircuit's public benchmarks. On those, TraceMaker now solves 188 of 200 boards of `srj19` (tscircuit's router: 184), 133 of 200 of `srj20` (tscircuit: 157, so the target is *not* met) and 60 of 72 `fanout31` cases (tscircuit: 48). The tier results of Section 5 were measured on release 0.8.0 and were not repeated on the new code; the sets that bound the router's behaviour on ordinary boards (30 boards of the `quick` tier and 11 of `mid`) give byte-identical output before and after the changes. The companion document `report/roadmap_testing.pdf` lists every milestone, test and benchmark run up to release 0.8.0.

1 What TraceMaker does

A printed circuit board (PCB) starts as a set of footprints with copper pads and a netlist that says which pads must be connected. The router must draw copper tracks (and vias between layers) that make every connection while respecting the board's design rules: minimum clearances between different nets, track widths, hole sizes, the board edge, solder-mask openings and more. A route is only useful if the board then passes the design-rule check (DRC) of the EDA tool the designer uses. TraceMaker therefore treats KiCad's DRC as the judge.

Figure 1 shows the task on a real board from the benchmark set: a three-channel motor driver with fine-pitch QFN packages. The left panel shows the input, with the required connections drawn as straight airwires. The right panel shows TraceMaker's result, which KiCad's DRC reports as complete with no violations.

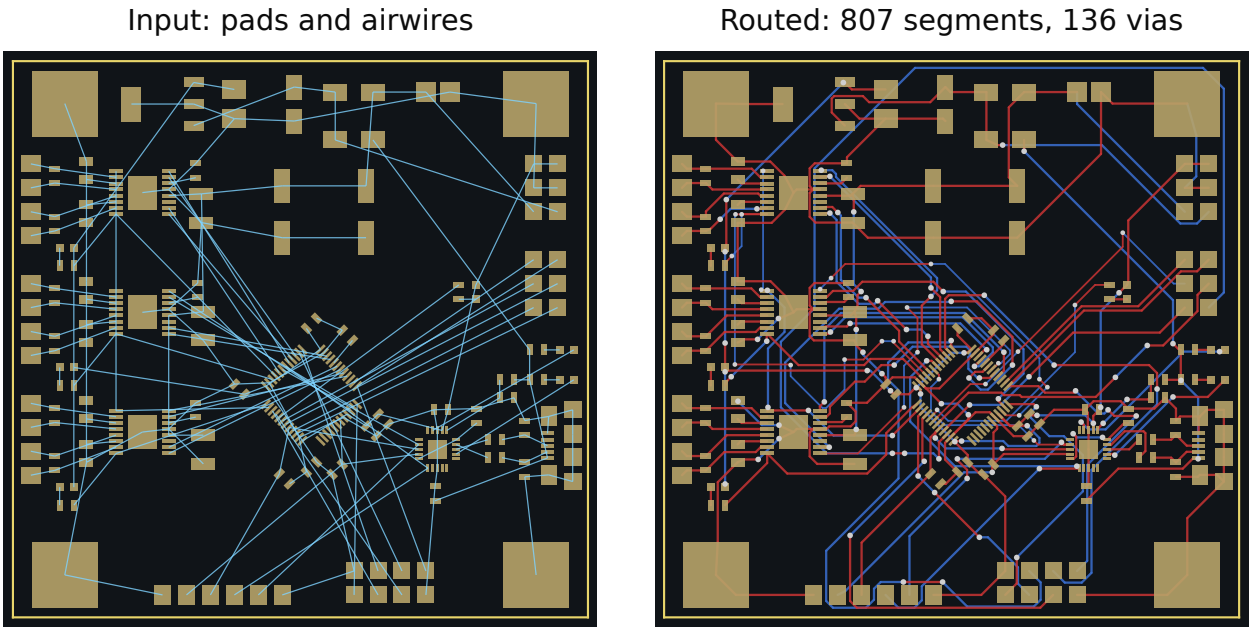


Figure 1: PCBench board `motor-3xdrv8833` (tier B, 195 connections). Left: input pads and airwires (a minimum spanning tree per net). Right: TraceMaker's routing on the front (red) and back (blue) copper, with vias in grey. KiCad's DRC: fully connected, no router-introduced violations.

2 Architecture

TraceMaker is about 12,500 lines of C++20 and CUDA, plus a 2,000-line TypeScript viewer and Python tooling. It runs on Ubuntu with an NVIDIA P100 and V100, but every GPU path has a CPU reference that gives identical results. Figure 2 shows the pipeline.

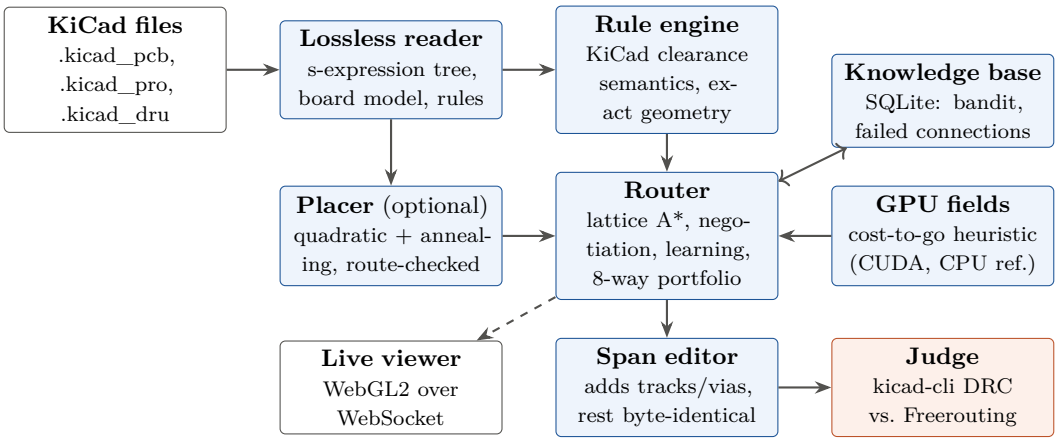


Figure 2: TraceMaker pipeline. The router never commits copper that the exact rule engine has not verified; the finished board is judged by KiCad's own DRC.

Lossless KiCad I/O. The reader parses the board into an s-expression tree that remembers the byte span of every node. Edits (adding a track, moving a footprint) replace or append spans; everything else is written back byte for byte. All 3,538 fixture boards in the test set round-trip identically. The reader handles KiCad 4 to 10 formats, including legacy arcs, numbered and named nets, custom layer names and project rule files.

Exact geometry. All coordinates are 64-bit integer nanometres, as in KiCad. Shapes are "rounded cores": a point, segment or polygon inflated by a radius, which represents round pads, tracks, vias and rounded rectangles exactly. Distance predicates use 128-bit integer arithmetic, so a clearance test never depends on floating-point rounding.

A KiCad-equivalent rule engine. Clearance is not a single number in KiCad. It is the maximum of the board minimum, the net-class clearances of both items, local pad and footprint overrides, diff-pair gaps and custom rules, with special cases for holes, the board edge and solder-mask openings. Each rule TraceMaker implements was checked against `kicad-cli` on real boards (Section 4). Examples found this way include: plated through-holes are on every copper layer whatever the file says; non-plated holes carry the board-edge clearance; a solder-mask opening exposes copper of its pad's net on both sides; mask graphics grow by the board's mask expansion; copper text is an obstacle.

3 How the router works

3.1 Search on an octilinear lattice

The board is covered by a lattice of points per copper layer. The pitch is a sixth of the smallest *track width + clearance* on the board (25–100 µm), so lattice tracks fit between fine-pitch pads. From each point a track may move in eight directions (0°, 45°, 90°, …) or change layer through a via. A connection is found with A* search, which expands lattice points in order of

$$f(n) = g(n) + h(n), \tag{1}$$

where $g(n)$ is the cost so far and $h(n)$ a lower bound on the remaining cost. The step cost is

$$c(u \to v) = \ell(u, v) \;+\; [\text{via}]\, c_{\text{via}} \;+\; [\text{bend}]\, c_{\text{bend}} \;+\; h_{\text{hist}}(v) \;+\; [\text{crosses other copper}]\, c_{\text{soft}}, \tag{2}$$

with track length ℓ, a via cost c_{via} (1 mm of track by default), an optional bend penalty, the congestion history h_{hist} of Section 3.2 and, during negotiation only, a cost for crossing other connections' copper. The heuristic h is the octile distance to the target or, on large search windows, an exact cost-to-go field computed on the GPU (Section 3.5).

Legality comes from the rule engine. Whether a lattice point is usable for a track of half-width $w/2$ on a given net is decided by the same rule engine as the DRC: the disk of radius $w/2$ around the point must keep the required clearance from every other net's copper, from holes, the board edge, keepouts, solder-mask openings and copper text. Results for fixed copper are cached per net class and width; routed copper sits in a separate spatial index, and a count raster lets the search skip that index far from any routed copper.

Nothing is committed without an exact check. A lattice path is converted to straight segments (merging collinear steps) and every segment and via is verified exactly against the rule engine before it is written. If the check fails, the offending lattice points are learned as blocked for that net and the search is repeated. Short "escape stubs" off the lattice connect fine-pitch pads whose centres do not sit on lattice points.

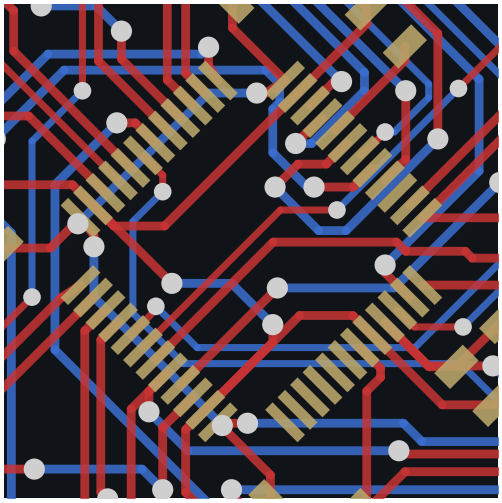
3.2 Learning from failure

The requirement was to "try the optimal first, store the failure information and use it to improve the next attempt". TraceMaker does this at four levels.

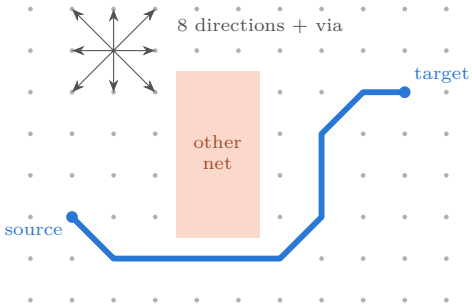
1. **Within a search.** A failed search reports *why*: the source or target pad is boxed in (the open list ran out without reaching the window edge), the window was too small, the search budget ran out, or the exact check rejected the path. A boxed-in target is detected with a short reverse search from the target. The reason picks the next step instead of blindly retrying.
2. **Escalation ladder per connection.** Strict routing first (no crossing of other copper); then forced off-lattice escapes for pads with no lattice exit; then a neck-down to the board's minimum track width; then negotiation.
3. **Negotiated rip-up (PathFinder).** In negotiation a path may cross other connections' copper at a cost. The crossed connections are ripped up and queued again, and every contested lattice point accumulates a history cost

   $$h_{\text{hist}}(v) \leftarrow h_{\text{hist}}(v) + 1 \quad \text{whenever } v \text{ is contested}, \tag{3}$$

   so that later searches avoid congested places. This is the PathFinder idea of McMurchie and Ebeling (1995). "Nogoods" record attempts that failed in exactly the same surroundings, so they are not repeated, and the best legal state seen so far is always kept. When negotiation stalls, the router restarts from an empty board with the hardest connections first, keeping the history; once the planned restarts are used, further restarts clear the nogoods, halve the history and shuffle ties while budget remains.



(a) Close-up of fine-pitch QFN escapes on the motor-driver board (14 mm square). Pads in gold.



(b) Octilinear lattice search: each point has eight neighbours on its layer plus a via move; points within clearance of another net's copper (shaded) are not usable.

Figure 3: Detailed routing on the lattice.

4. **Across runs.** A SQLite knowledge base stores, per class of board, how well each router variant did and which connections failed on each board. A Thompson-sampling bandit chooses variants: each variant k has a Beta(α_k, β_k) posterior over its success rate, a sample θ_k is drawn for each, and the variants with the largest samples run. Connections that failed before are routed first next time.

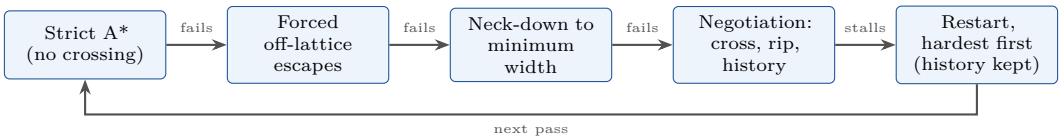


Figure 4: Escalation ladder for a connection that does not route. Each failure is classified, and the classification decides the next rung.

3.3 Clean-up after routing

Search on a lattice finds legal paths, but not tidy ones: equal-cost staircases of short 45° steps, and vias placed where a slightly longer single-layer route existed. Once routing is done, TraceMaker therefore cleans up, much as Freerouting's optimizer does:

1. **Via-saving re-routes.** Connections that use vias are taken out one at a time, most vias first, and routed again with vias made ten times dearer and other connections' copper as a hard obstacle. The new route is kept only if it is cheaper by track length plus 2 mm per via; otherwise the old copper is restored exactly. Rounds repeat while connections improve and search budget remains.
2. **Path smoothing ("pull tight").** Within each connection, runs of same-layer, same-width segments are replaced by the fewest octilinear segments that the exact rule check accepts, keeping pads and vias where they are. This step is pure geometry, cheap and deterministic, so it always runs.

Every new segment passes the same exact check as during routing, so clean-up cannot introduce a DRC error.

3.4 Parallel portfolio and determinism

Different strategies suit different boards: short connections first or long first, cheap or expensive vias, exact bend tracking or fast bends, on large boards a lattice of double pitch and on small boards one of half pitch. TraceMaker runs a portfolio of eight such variants in parallel threads and keeps the best result (most connections, then fewest vias, then shortest copper). When one variant has routed everything after time T, the others may continue until $2T + 5$ s, so a cleaner complete result can still win.

Runs can be made fully deterministic with a work budget (`--work`, counted in A* expansions) instead of a time limit: random choices use a counter-based Philox generator keyed by (seed, stage, item), hash

containers are never iterated in an output-affecting order, and costs are integers. Repeated runs then produce byte-identical boards at any thread count, which the regression tests rely on.

3.5 GPU cost-to-go fields

On large search windows A* spends most of its time exploring around obstacles that a straight-line estimate does not see. Before such a search, a CUDA kernel computes the exact obstacle-aware cost to the target from every lattice point of the window (iterated GAMER-style sweeps over the grid). This field is a lower bound that is much tighter than the octile distance, so A* expands far fewer points. The field never prunes: where it claims "unreachable", the router falls back to the octile bound, so the search stays complete. A CPU implementation produces bit-identical fields, and the tests compare them. Because the GPUs are shared with other jobs, a variant stops building fields if they take more than 30% of its wall-clock time.

3.6 Component placement

Placement (`tracemaker-place`) works in stages: a quadratic wirelength minimisation with the bound-to-bound net model, solved by conjugate gradients; SimPL-style spreading to remove overlap; rotation choice; legalisation to the nearest legal position (courtyards, board outline, copper and solder-mask clearances); and parallel simulated annealing on half-perimeter wirelength

$$\mathrm{HPWL} = \sum_{\text{nets } e} \left(\max_{p \in e} x_p - \min_{p \in e} x_p + \max_{p \in e} y_p - \min_{p \in e} y_p \right) \tag{4}$$

plus a penalty per airwire crossing. A linear-programming lower bound on HPWL, solved as a min-cost flow, is reported alongside so the quality of each placement can be judged.

Shorter wiring does not automatically mean a board routes better: denser packing can block escape routes. The placer therefore puts the router in the loop. With `--route-check`, the input placement and the new one are routed with the same deterministic budget, and the new placement is kept only if it leaves no more connections unrouted. `--mode auto` does this for both the refine and the full mode and keeps the best of the three.

3.7 Live view and KiCad integration

`tracemaker route --view` streams the routing as it happens to a WebGL2 viewer in the browser: tracks appearing and being ripped up, the A* frontier, failures and the remaining airwires. A KiCad 10 action plugin routes the open board through KiCad's IPC API and adds the result in one undoable commit; it can also re-route existing unlocked copper, move footprints first and refill zones.

4 How it was tested

- **Unit and integration tests** (262 in the current code, run with `ctest`; release 0.8.0 had 132): exact unit conversion, Philox known-answer vectors, GPU-equals-CPU for the random generator and cost-to-go fields, s-expression parsing and byte-exact round trips, the board reader against KiCad's own model, rule reading, geometry predicates and the indexed paths against their linear references, placement legality and lower bounds, the component-rule catalogue and impedance formulas, differential-pair geometry, and the viewer server and replay log, and since 0.8.0 the Simple Route JSON importer, bus routing and the after-route passes. The suite passes in the release build (262 of 262 on 11 October, after merging main). The address-sanitizer build compiled and passed all tests it finished; `bus_route` and `thread_determinism` are too slow under it and were not run to completion for this edition.
- **DRC parity.** TraceMaker's own DRC is compared with `kicad-cli` on the KiCad demo boards: 14 of 17 match exactly on the 15 routing-relevant violation types, and 264 of 264 boards with injected defects match violation by violation; both are regression tests. The router uses the same rule engine, so parity problems show up as router errors and were fixed at the source.
- **Benchmark harness.** `bench/run.py` samples boards from Freerouting's PCBench set by Freerouting's own difficulty tiers, routes each with a snapshot of the binary (120 s, eight variants), and runs `kicad-cli pcb drc` on the result. A board is *clean* when it is fully connected and KiCad reports no violation involving a track, via or arc that was not already present in the unrouted board. The result is compared with Freerouting's published per-board results for the same files.

- **Determinism.** With a work budget the output is byte-identical at any thread count and with or without a GPU; a test routes the same board at one, three and eight threads and compares the files.
- **A running KiCad.** The plugin is driven against `pcbnew` on a virtual display: copper appears in the open board, one undo removes it, and KiCad's DRC of the saved board shows no new error.

5 Results

5.1 Clean pass by tier

Figure 5 and Table 1 give the final run (`final11`, release 0.8.0). TraceMaker matches Freerouting 2.5.0-RC12 on the routine tier-A boards and is ahead on the harder tiers. No board has a router-introduced DRC error.

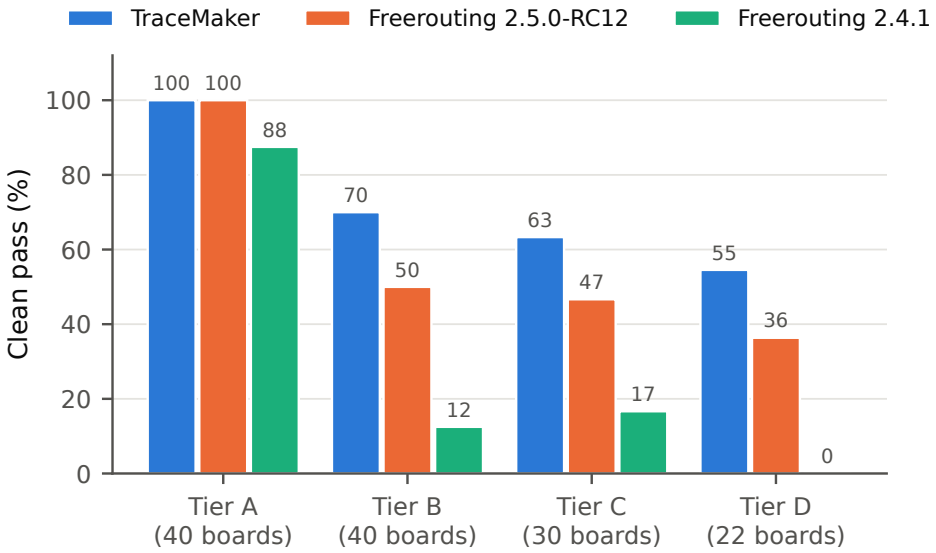


Figure 5: Share of boards routed clean (fully connected, no router-introduced KiCad DRC errors), 120 s per board. Freerouting figures are its published results on the same fixture files.

Tier	Boards	TraceMaker	FR 2.5.0-RC12	FR 2.4.1	Router errors
A (routine 2-layer)	40	**100%**	100%	87.5%	0
B (2–4 layer)	40	**70.0%**	50.0%	12.5%	0
C (complex)	30	**63.3%**	46.7%	16.7%	0
D (hardest)	22	**54.5%**	36.4%	0.0%	0

Table 1: Final benchmark run. Samples: 40 of 453 tier-A boards, 40 of 560 tier-B, 30 of 122 tier-C and all 22 tier-D boards (fixed seed). FR: Freerouting's published results. Router errors: boards with router-introduced DRC errors. The four tiers of this run were routed at the same time (64 router threads on 56 cores). The first version of this report gave 100, 65.0, 60.0 and 54.5 % (`final9`).

Figure 6 compares the two routers board by board. On every tier there are more boards that only TraceMaker routes clean than boards that only Freerouting does (tier B: 10 against 2; tier C: 7 against 2; tier D: 5 against 1).

5.2 Progress during development

Figure 7 tracks every full benchmark run since the project began. The first router (plain lattice A*) routed 70% of tier A clean. Fixing rule semantics found against KiCad, the learning mechanisms and the escalation ladder took tier A to 100% within about two and a half hours; the later gains on tiers B–D came from boxed-in detection at both ends of a connection, pad-approach fixes for fine-pitch parts, the coarse-lattice variants for large boards, extra restarts and the large-board setup fix. Since then the curves are nearly flat: the work of 4–6 October (Section 6) added features and fixed faults more than it moved clean pass.

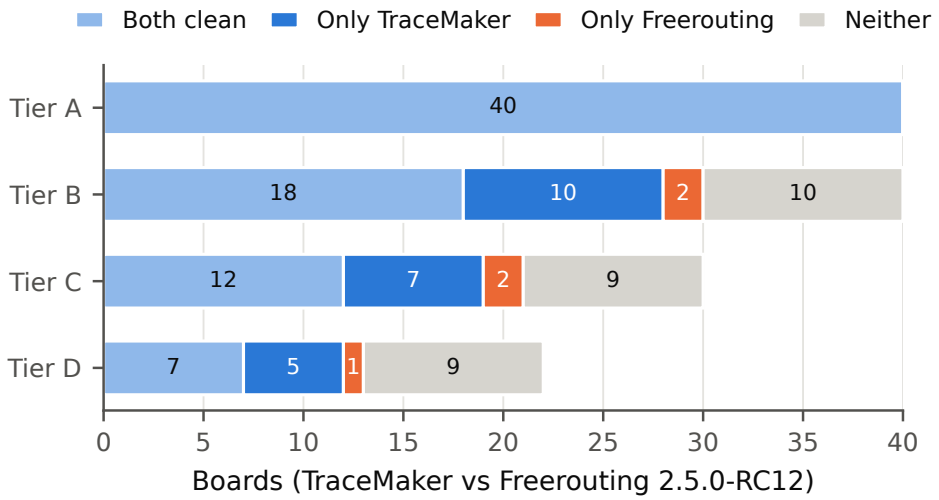


Figure 6: Board-by-board comparison with Freerouting 2.5.0-RC12 in the final run.

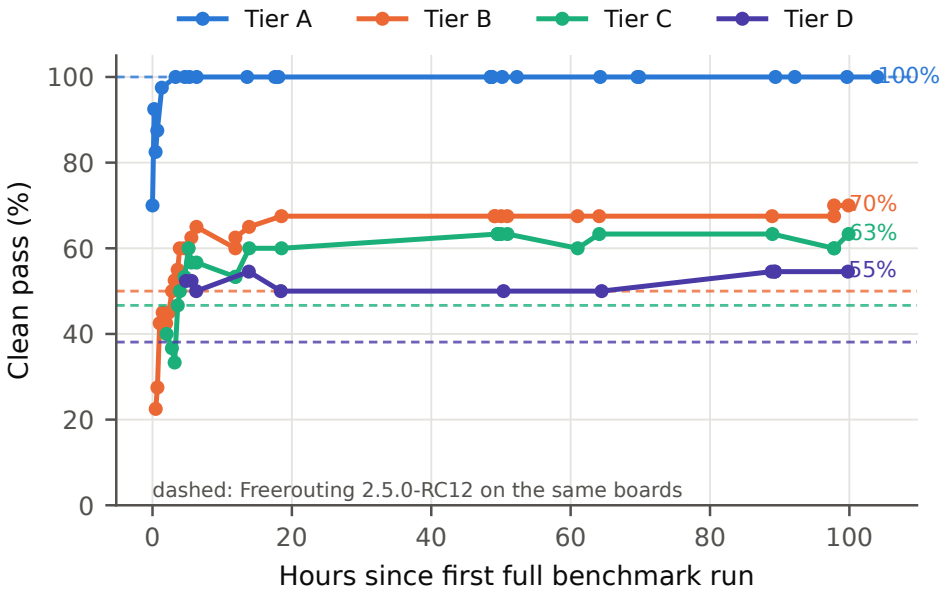


Figure 7: Clean pass of every full benchmark run (20 or more boards) since the first one, including side-by-side experiment arms. Dashed lines: Freerouting 2.5.0-RC12 on the same boards. Small run-to-run differences come from the wall-clock limit.

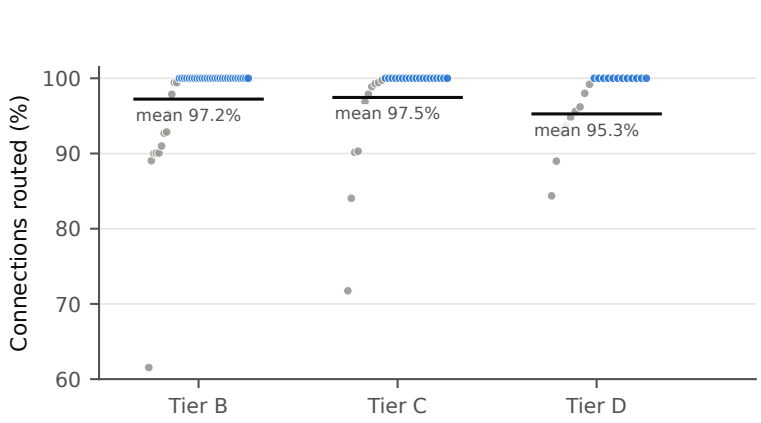
5.3 Completion and speed

Clean pass is all-or-nothing. Figure 8a shows how close the remaining boards are: most boards that are not clean miss only a few connections, and mean completion is above 95% on every tier (97.2, 97.5 and 95.3 % on tiers B, C and D).

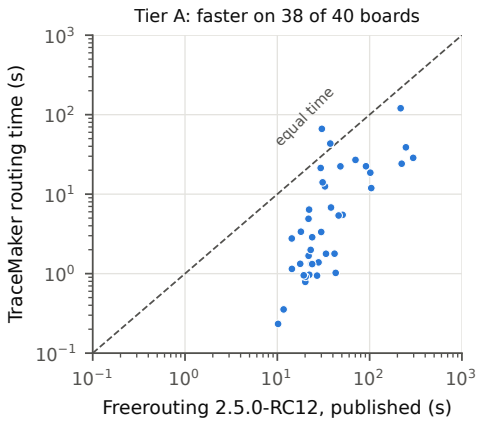
On tier A, routed on its own with the release 0.8.0 binary (`final12-tierA`), the median TraceMaker routing time is 3.4 s per board (11.4 s including file input and output and the judge's set-up), and 38 of the 40 boards finished faster than Freerouting's published time (Figure 8b). The first version of this report measured 2.5 s: the portfolio variant that routes small boards at half the lattice pitch does four times the work on them, and the others wait for it up to the grace period.

5.4 Routed boards

Figure 9 shows a selection of routed boards from the final run, rendered from the output files. The four-layer high-voltage switching board (604 connections) uses the inner layers (yellow and green); `draco` and `EUC-VESC` are boards that TraceMaker routes clean and Freerouting does not.



(a) Share of connections routed per board. Blue: fully routed.



(b) Tier-A routing time against Freerouting's published time.

Figure 8: Completion on the harder tiers and speed on tier A. Freerouting's times were measured on its own hardware, so the comparison is indicative only; TraceMaker used eight threads of a 56-core server.

5.5 Placement

The placer was evaluated on 23 PCBench boards with a deterministic router budget of three million A* expansions per variant (Figure 10, Table 2). In `--mode auto` it kept a new placement on 17 boards and the human placement on six: total HPWL fell from 17,464 mm to 13,360 mm (−23.5%, median −17% per board), unrouted connections fell from 64 to 60, and no board got worse. The `--mode routable` loop (routability term in the annealer, parallel tempering, large-neighbourhood search and exact windows, re-placing the parts around failed connections between routing rounds) brought unrouted connections down to 35, with 19 boards fully routed against 16 for the human placement; `--mode eco` (a few legal moves near failed connections on the human placement) reached 48. No mode added a KiCad DRC error. Without any router check, placement alone cut wirelength by a similar amount but routed worse: denser packing can block routing, which is why the checks exist.

	Human	`auto`	`routable`	`eco`
Unrouted connections (sum)	64	60	**35**	48
Boards fully routed	16	17	**19**	19
Total HPWL (mm)	17,464	13,360	**12,403**	17,433
New KiCad DRC errors	–	0	0	0
Placement wall time, all boards (s)	–	1,354	3,481	1,541

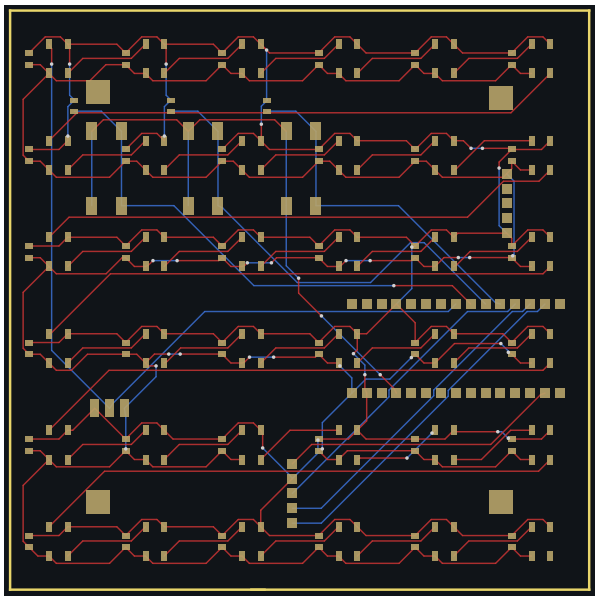
Table 2: Placement modes on the 23 development boards (deterministic router budget). These boards were used while developing the placer, so the gains are optimistic; the held-out evaluation is in Section 6. An earlier version of this table was affected by a bug: when the router check rejected a placement, the edited board was written instead of the input.

5.6 Quality benchmark on held-out boards

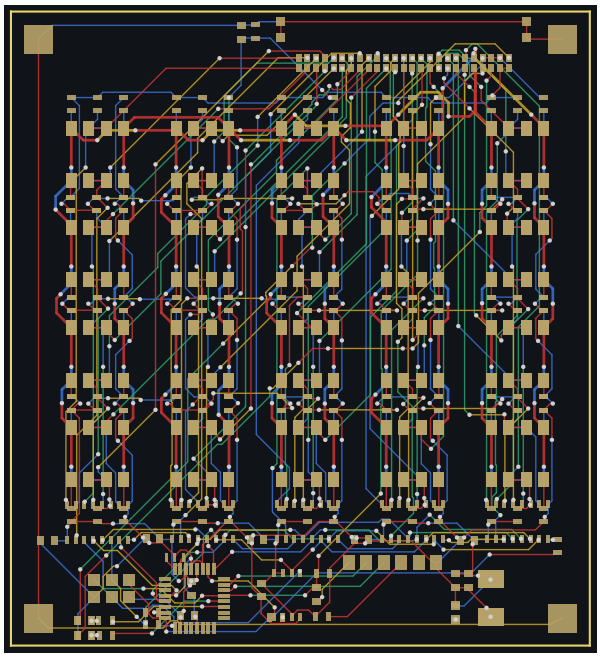
The tier results above come from boards that were used while TraceMaker was developed, and compare against Freerouting's published numbers. To remove both weaknesses, 60 PCBench boards that never appeared in any development run (20 each from tiers A, B and C, fixed random seed) were routed by TraceMaker, Freerouting 2.5.0-RC12 and Freerouting 1.9.0 on the same machine. Freerouting used its published settings (one thread, 500 passes, fanout and optimizer on, 30-minute limit) on the fixture's Specctra file, and its result was imported into the KiCad board. ("2.5.0-RC12" is the name of the jar in Freerouting's own benchmark set; the program reports itself as v2.4.2-SNAPSHOT, built 2026-09-29.) Every result was judged by KiCad's DRC and measured by the same quality script (`bench/quality.py`); the human-routed originals were measured as a reference.

What the held-out benchmark shows:

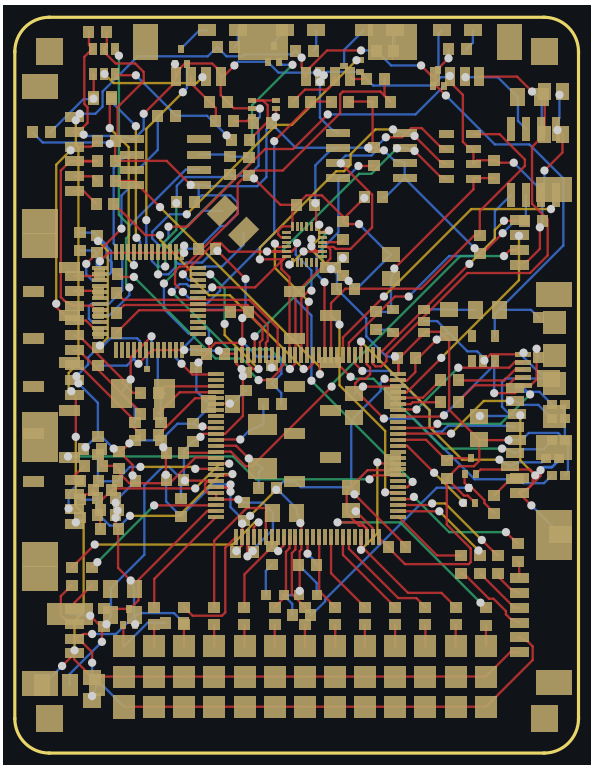
- **Correctness is TraceMaker's clearest advantage.** No TraceMaker result had a router-introduced KiCad error. Freerouting connects about as many boards, but almost none of its results pass KiCad's DRC. On 38 of the 60 boards Freerouting 2.5's only errors are board-edge clearance or tracks below the



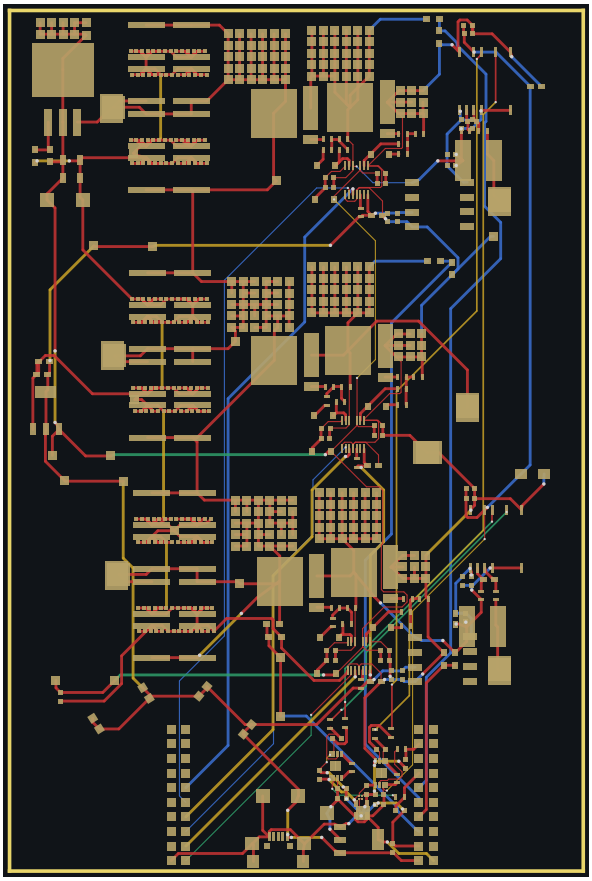
(a) `Hangul-Clock` (tier A): 209 connections, 39 s.



(b) `40-channel-hv-switching-board` (tier C, 4 layers): 604 connections.



(c) `draco` (tier C): 418 connections; clean only with TraceMaker.



(d) `EUC-VESC` (tier D): 760 connections; clean only with TraceMaker.

Figure 9: Boards routed by TraceMaker in the final benchmark run, each fully connected with no router-introduced KiCad DRC errors. Red: front copper; blue: back copper; yellow and green: inner layers; gold: pads; grey: vias.

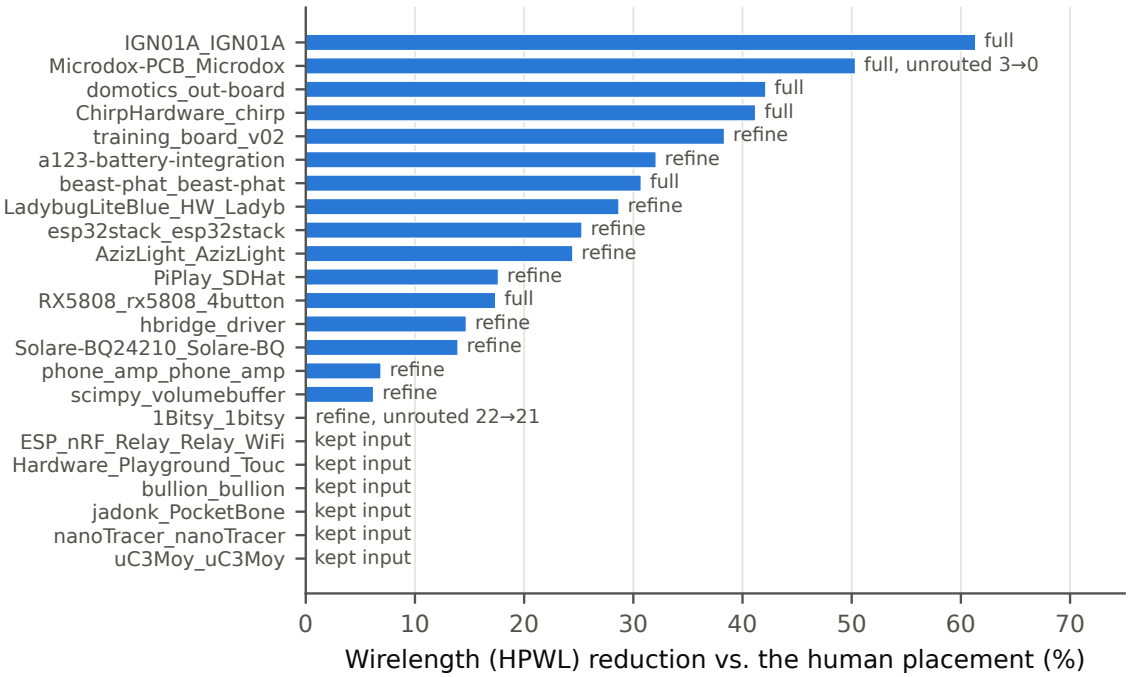


Figure 10: Wirelength reduction of the placement kept by `tracemaker-place --mode auto`, per board. "Kept input": the router check kept the human placement. Labels give the mode kept and any change in unrouted connections.

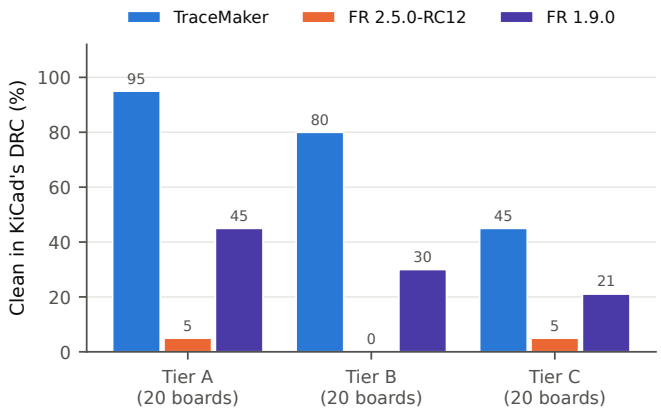
Router	KiCad clean	Connected	Completion	Router errors	Median time
TraceMaker	**73%**	73%	98.3%	**0**	38 s
Freerouting 2.5.0-RC12	3%	78%	97.4%	58	24 s
Freerouting 1.9.0	32%	66%	96.8%	34	191 s
Human original	3%	77%	98.0%	58	–

Table 3: Held-out quality benchmark, 60 boards (`quality-3`). Three TraceMaker runs of the same boards gave 72–78% clean, so differences of a few points are within run-to-run variation. "Router errors": boards with router-introduced KiCad DRC errors; TraceMaker includes the clean-up stage. Freerouting 1.9.0 failed to produce a result on one board.

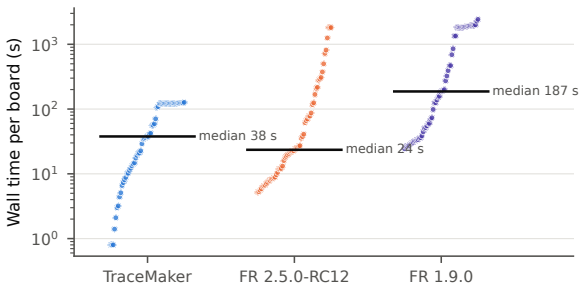
minimum width, rules that KiCad's Specctra export does not pass on; both Freerouting versions also had genuine clearance errors on 9–10 boards and shorts on 3–4.

- **Connection rate is about equal.** TraceMaker fully connected 73–78% of boards across runs, Freerouting 2.5 78% and Freerouting 1.9 66%.
- **Layout quality is mixed.** Against Freerouting 2.5, TraceMaker's tracks are 5% shorter and it has 28% fewer bends, but 43% more vias (medians over 38 boards both completed). Against Freerouting 1.9 its tracks are 4% shorter with 22% fewer bends, but 2.2 times the vias. The clean-up stage cut the via ratio from 2.15 to 1.43 and the bend ratio from 1.20 to 0.72 against Freerouting 2.5.
- **Acute angles:** TraceMaker had none outside pad copper on any board; Freerouting 2.5 had 9 and Freerouting 1.9 had 7 in total.
- **The held-out set found two real bugs.** Text drawn on a solder-mask layer opens the mask; routing under it caused bridges on two boards before the fix.

More held-out boards. Two further sets were run the same way after the first: 30 more tier-C boards never used in development, and the DAC 2020 PCB benchmarks (10 boards that have Specctra files; the authors published them as layout benchmarks, with their human routing, which was stripped for the test). Freerouting 2.5 fully connects more of these boards than TraceMaker, but almost none of its results pass KiCad's DRC; TraceMaker's results that connect everything are clean. The DAC 2020 run also found a TraceMaker bug: solder-mask openings drawn by a footprint over its own pads were treated as walls, which left two boards at 37% and 60% completion; after the fix both routed completely and clean.



(a) Clean in KiCad's DRC, by tier.



(b) Wall time per board (recorded, not optimised for).

Figure 11: Held-out boards: correctness and time.

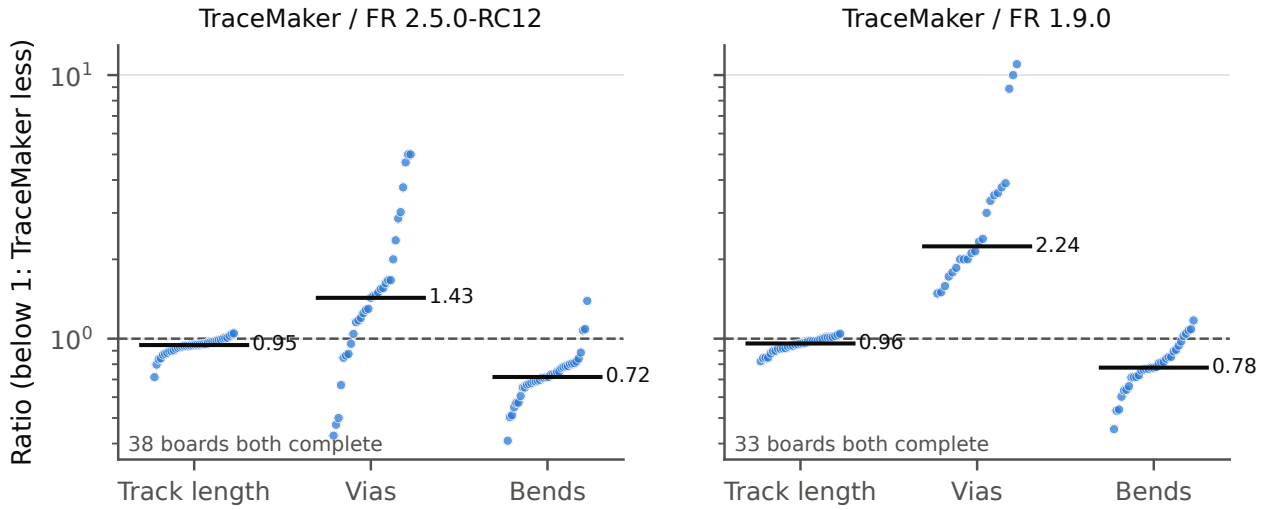


Figure 12: Layout quality on boards that both routers fully connected: TraceMaker's value divided by Freerouting's, one dot per board; the bar is the median.

5.7 Head to head on one board

To compare layout quality and not only pass or fail, the tier-C board `AmpOne` (a two-layer audio amplifier, 321 connections) was routed by TraceMaker, Freerouting 2.5.0-RC12 and Freerouting 1.9.0 on the same machine, and measured together with the human-routed original (Figures 13 and 14, Table 5). Freerouting ran with its published benchmark settings (one thread, up to 500 passes, fanout and optimizer on, 30-minute limit) from the fixture's Specctra file, and its session was imported back into the KiCad board, so KiCad's DRC and the same quality metrics judge every result.

A video of all three routers working on this board, played at 4× real time, is in `report/compare_AmpOne.mp4`. What the comparison shows:

- **All three routers connect everything; only TraceMaker passes KiCad's DRC.** Freerouting 2.5.0-RC12 leaves three hole-clearance and four board-edge-clearance errors, and Freerouting 1.9.0 seven board-edge-clearance errors. Both report their own result clean: the Specctra file they route from does not carry all of KiCad's rules.
- **TraceMaker uses the least copper and the fewest bends.** Its tracks are 4% shorter than Freerouting 2.5's, 6% shorter than Freerouting 1.9's and 18% shorter than the human routing, with 594 bends against 723 and 610.
- **Freerouting uses fewer vias.** TraceMaker uses 59 vias, Freerouting 2.5 42 and Freerouting 1.9 23. Before the clean-up stage (Section 3.3) TraceMaker used 90 vias and about 1,100 bends on this board. TraceMaker joins 20 track pairs at acute angles at pad centres, where the pad's copper fills the angle, and has none outside a pad (Freerouting 1.9 has one, the human routing nine). It narrows 97 mm of track below the design width to pass tight spots.

Set	Router	KiCad clean	Connected	Completion	Router errors
Tier C, 30 more	TraceMaker	**43%**	43%	96.5%	**0**
	Freerouting 2.5.0-RC12	0%	57%	95.5%	30
	Freerouting 1.9.0	21%	41%	94.9%	17
	Human original	13%	97%	100%	26
DAC 2020, 10	TraceMaker	**60%**	60%	96.7%	**0**
	Freerouting 2.5.0-RC12	10%	80%	98.9%	9
	Freerouting 1.9.0	20%	50%	93.8%	5
	Human original	60%	100%	100%	4

Table 4: Further held-out sets. "Router errors": boards with router-introduced KiCad DRC errors (for the human original: errors in the original design). On boards both routers completed, TraceMaker's tracks were 7% shorter than Freerouting 2.5's with 25–31% fewer bends, but 1.5–2 times the vias.

	Human original	TraceMaker	FR 2.5.0-RC12	FR 1.9.0
Connections routed	100.0%	100.0%	100.0%	100.0%
Router-introduced DRC errors	n/a	0	7	7
Track length (mm)	3,179	2,607	2,714	2,772
Length / pad MST	1.43	1.17	1.22	1.25
Vias	258	59	42	23
Bends	669	594	723	610
Acute angles outside pads	9	0	0	1
Acute angles at pad centres	8	20	0	0
Narrowed track (mm)	2.0	96.7	40.2	0.0
Wall time (s)	–	78	39	394

Table 5: Quality metrics for `AmpOne`. "Length / pad MST" divides the track length by the length of a minimum spanning tree over each net's pad centres. "Narrowed" is track thinner than the net's design width. Acute angles count joints of two tracks of the same net at less than 90°; only those outside pad copper are etchant traps. The human original is shown for reference: it does not pass the board's own rules (371 clearance errors).

- **Speed:** TraceMaker 78 s with eight threads, Freerouting 2.5 39 s and Freerouting 1.9 394 s, each with one thread.

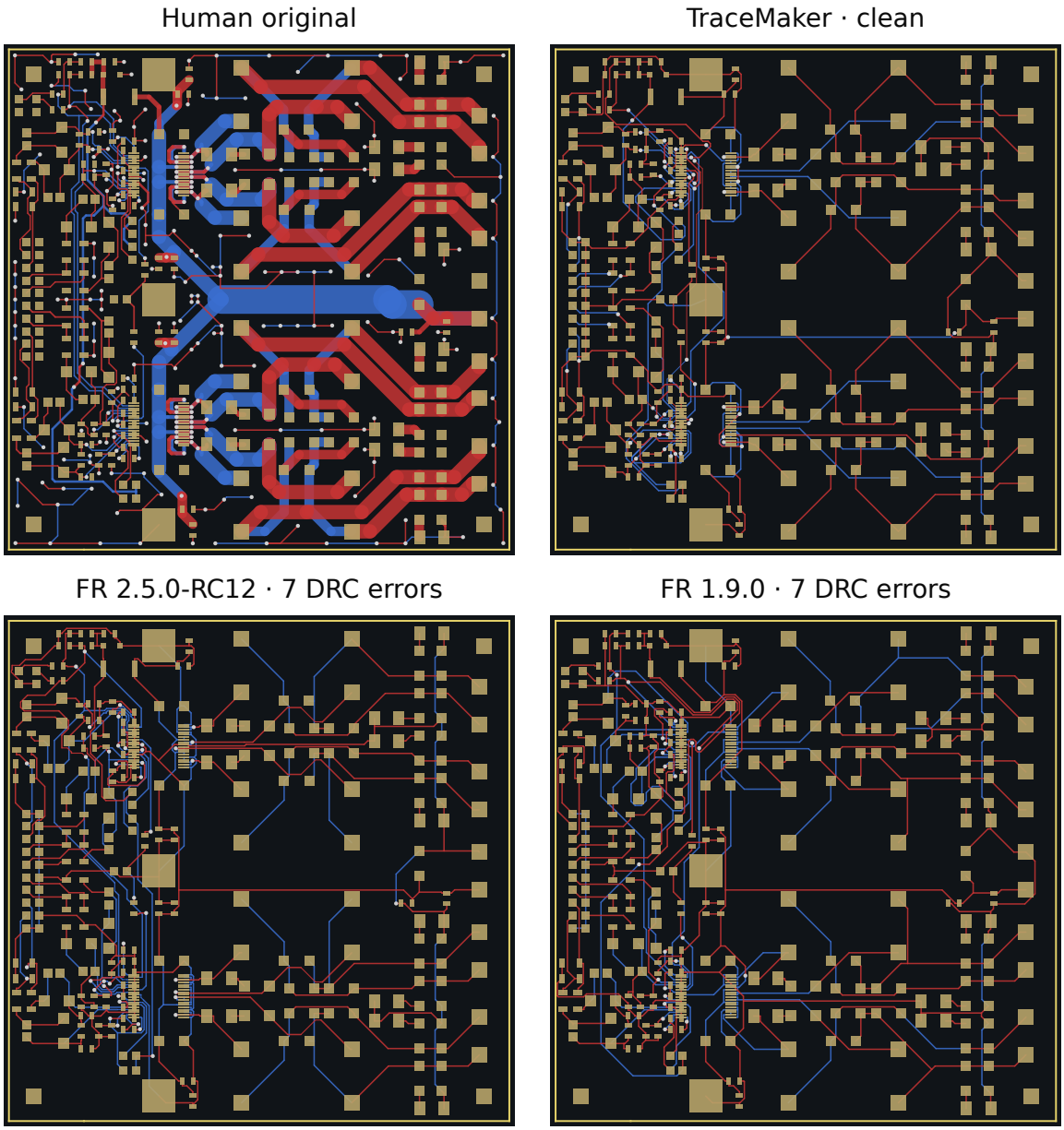


Figure 13: `AmpOne` routed four ways. The human design uses very wide power tracks; none of the routers does, because the board's net classes do not ask for them.

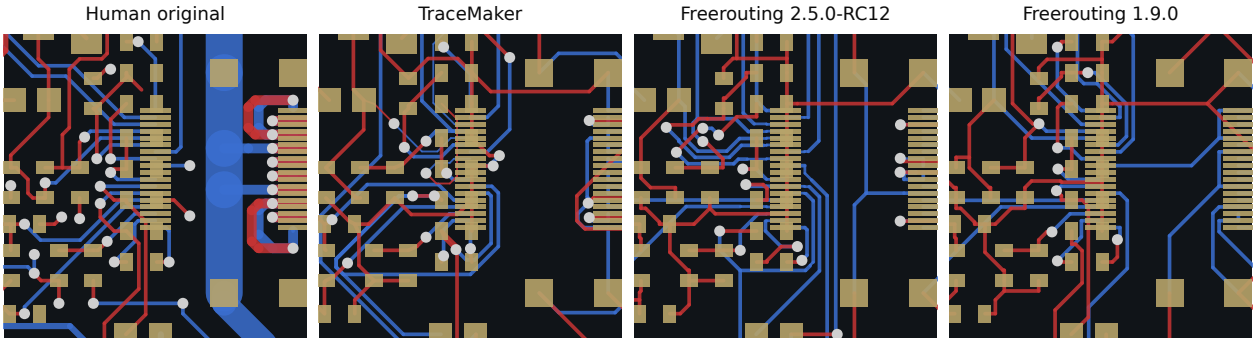


Figure 14: The same 22 mm window, centred on the densest pad area, in each result.

6 What was added after the first version

The first version of this report described the state on 2–3 October. This section summarises what was built and measured between then and release 0.8.0; `report/roadmap_testing.pdf` has the numbers behind each item.

Routing features.

- **Escape planning** for dense packages: reserved exit corridors for perimeter pins and dog-bone sites for inner balls, used by two of the eight portfolio variants, and `tracemaker escape`, which reports pins that cannot leave their package under the board's rules. A flow-based planner for deep arrays was built and measured below the simple corridors; it is opt-in.
- **Differential pairs** routed as one object: a coupled search with coupled vias, breakout legs, re-coupling after rip-up, skew tuning, and twists (a layer change with the sides swapped) for pairs whose pin order is mirrored between the ends. On 18 boards with 59 pairs, 36 are coupled over at least half their length; the median coupled share is 63 %.
- **Length tuning** to KiCad `length` rules by meanders; blind, buried and micro vias where the board's rules allow them.
- **Connection planning by copper distance:** each net's spanning tree weighs pads by the gap between their copper, not their centres.
- **A reachability pre-check:** an exact flood fill proves a search hopeless before the A* runs.

Component-aware rules. TraceMaker recognises common parts (USB, Ethernet, crystals, regulators and others) from footprints and nets, and applies a catalogue of 193 layout rules drawn from application notes: proximity pulls in placement (on by default), generated keep-outs, and with `--component-rules on` net classes whose widths come from impedance targets and the board's stackup or from current.

Placement. Parts can be moved to the other side of the board (opt-in), with mirroring that matches KiCad's exactly. For boards with no placement at all, `--scratch` places every movable part from a pile. This was measured on a held-out set built for the purpose: 40 PCBench boards never used before, with their movable parts piled at the board centre.

	Boards clean
Designer's placement, routed by TraceMaker	38 of 40
Placed from scratch, as the placer was on 3 October	28 of 40
Placed from scratch, release 0.8.0	**34 of 40 (85 %)**

Table 6: Held-out placement boards (eight router variants, 120 s, KiCad's DRC). The project's own target was 90 %; it is not met. Three of the six failing boards have a part that fits nowhere without breaking rules that the designer's placement itself breaks, and one cannot be routed clean in the designer's placement either.

Correctness and speed. Two faults were found by those held-out boards and fixed: copper had been routed across lines on the Margin layer, which KiCad treats as the board edge; and pads without a hole that are listed on both sides had been connected on the far side, which KiCad reports as unconnected. TraceMaker's DRC now tests copper against zone fills through an edge index: the check of the largest demo board fell from 730 s to 7 s with a byte-identical report.

KiCad integration. The plugin was run against a live `pcbnew` on a virtual display, and its package was installed through KiCad's Plugin and Content Manager (Section 4).

What did not help. Global routing was built twice (corridors, then a congestion map with net-shared edges and via capacity) and measured on 18 hard boards at a fixed budget: corridors cost completion (13,305 connections without, 12,967 with) and the map was neutral. A joint re-route of small groups of conflicting nets changed no board. Successive halving of the portfolio's budget was slightly worse than running the first variants. All three are off or removed.

What the benchmark measures. PCBench boards come without their project files, so KiCad's default rules judge them. Under that judge only 6 of the 92 designer-routed boards of tiers B to D are clean, and on 82 of them the designer used copper pours that the unrouted fixture no longer has. No straight line across any of those boards is more than 40 % full: the boards are not short of space, and what fails is the fan-out at a fine-pitch package under rules coarser than the original design's.

7 BGA breakout and SoC fan-out (7–11 October)

The task. tscircuit, an open-source electronics toolkit, publishes the boards its autorouter is tested against. Two sets are ball-grid-array breakouts on two copper layers: `srj19` (200 boards, MIT licence) and `srj20` (200 boards with some balls missing, no licence, used locally only). A third, `fanout31`, has 72 fan-out cases of six SoC packages (AM62L, RK3308, K230, i.MX6ULL, T113-S3, AM3352) on 8–10 layers with plane terminations, buses and skew goals. The goal was to route at least as many of them as tscircuit's router does, without adding a design-rule error to any board. Their published figures (dashboard run `38105514608-1`) are 92.0 % of `srj19`, 78.5 % of `srj20` and 48 of 72 `fanout31` cases.

How it was judged. Each problem is imported as a self-contained KiCad board (`tracemaker srj-import`), routed by the ordinary pipeline, and judged by `kicad-cli pcb drc` at the distances of tscircuit's own pass criterion (0.1 mm copper clearance, 0.1 mm between via holes). The routed copper can be exported back to the Simple Route JSON format (`srj-export`), where tscircuit's checker gives a second score. A board counts as solved only when it is fully connected with no router-introduced violation. All work budgets are counted in search steps, so a run is reproducible and independent of the machine.

7.1 Research before building

The plan (`docs/16-bga-breakout-plan.md`) and the evidence behind it (`research/bga-routing-2026-10.md`) rest on the literature on escape routing and on tscircuit's own public code and benchmark data. Three points shaped the work. First, the gap between the two routers is a cliff: boards with up to about 40 connections are solved by both, and the difference sits in boards of 50–79 connections. Second, escape feasibility for each ball separately does not imply that all balls can leave together, which is why exact methods for ordered escape routing (SAT, flow models with diagonal capacity) exist. Third, tscircuit's router uses many vias outside the ball array. The research note also catalogues approaches reported not to work (12 from the literature, 8 from tscircuit's history, 8 from TraceMaker's earlier milestones), so they were not repeated.

7.2 Results

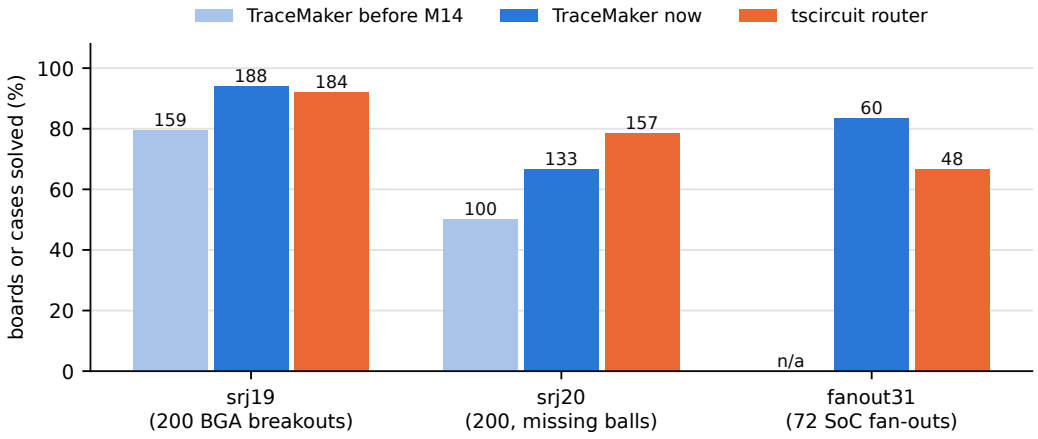
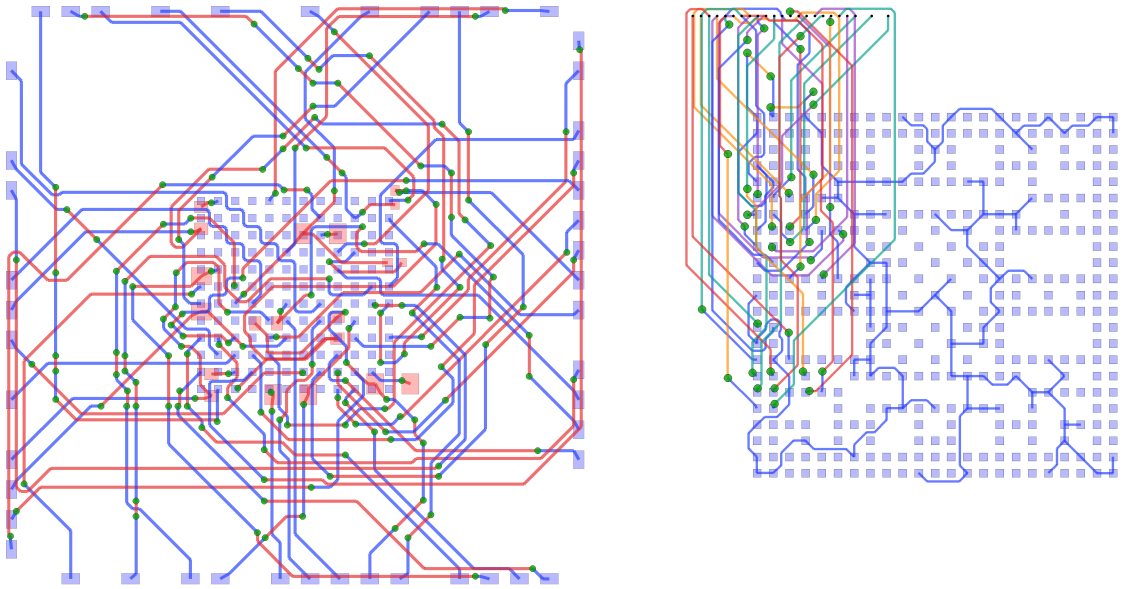


Figure 15: Boards solved in each tscircuit benchmark set. "Before M14": one variant, 50 million work units, as at the start of the milestone (no `fanout31` importer existed). "Now" is a composite: one variant on all boards, plus a 16-variant portfolio on the boards that one variant misses. tscircuit: its published dashboard run.

Set	tscircuit	TraceMaker now	Target	Met
`srj19`	184 of 200 (92.0 %)	**188 (94.0 %)**	≥184	yes
`srj20`	157 of 200 (78.5 %)	133 (66.5 %)	≥157	**no, 24 short**
`fanout31`	48 of 72	**60**	≥48	yes
KiCad errors added		0 on every run	none	yes
`quick` (30) and `mid` (11)		byte-identical	unchanged	yes

Table 7: Gate status on 11 October. `srj19`: 181 boards with one variant (50 million work units, fast bends, automatic breakout settings) plus 7 of the 19 that a 16-variant portfolio closes. `srj20`: 124 with one variant plus 9 of the 76 that the portfolio closes (median 168 s per board). The composite is not a single run.

`fanout31`, case by case. AM62L, RK3308, K230 and i.MX6ULL are solved in all 12 cases each; T113-S3 in 9 of 12 (cases 55–57 keep three connections open); AM3352 in 3 of 12 (the left-edge cases 70–72, with 16 variants; tscircuit solves none of the AM3352 cases). One variant solves 44 of the first 59 cases and an 8-variant portfolio closes the stragglers.



(a) `srj19` sample 196: 68 connections, two layers.

(b) `fanout31` AM62L top-centre: 133 connections.

Figure 16: Boards routed by TraceMaker (TraceMaker's renderer, one colour per copper layer, vias dotted).

7.3 What worked

- **Higher rip and pass limits.** Allowing each connection to be ripped up 40 times and the negotiation 40 passes (were 8 and 12) raised the clean share of the 24 `srj19` boards that one variant first missed from 33 % to 54 % at equal work. These are now the defaults; `quick` and `mid` are byte-identical.
- **Fast bends** (cheaper search around bends): at equal wall time 81.7 % to 85.9 % of connections routed on the same boards.
- **Preferred layer directions and a package penalty.** Orthogonal steps against a layer's preferred direction (even layers horizontal, odd layers vertical) cost extra, and tracks on the pad layer inside a ball array cost more. This was the first structural gain on `srj20` (Figure 18a). On ordinary two-layer boards the same settings cost one to three points of completion (`quick`: 89.2 to 86.8 %), so they are applied automatically only to a *breakout board*: two copper layers, at least 70 % of the connections ending on a ball-grid-like package (at least 30 same-layer SMD pads on at least 5×5 positions with 40 % of the grid filled). A first version of this test also matched the connectors of a bus-routing test board and broke it; requiring the grid shape fixed that.
- **A larger portfolio.** Beyond the designed eight variants, `--variants N` adds seeded variants with different orders and via and crossing costs. Sixteen variants close 7 of 19 `srj19` boards and 9 of 76 `srj20` boards that one variant misses.

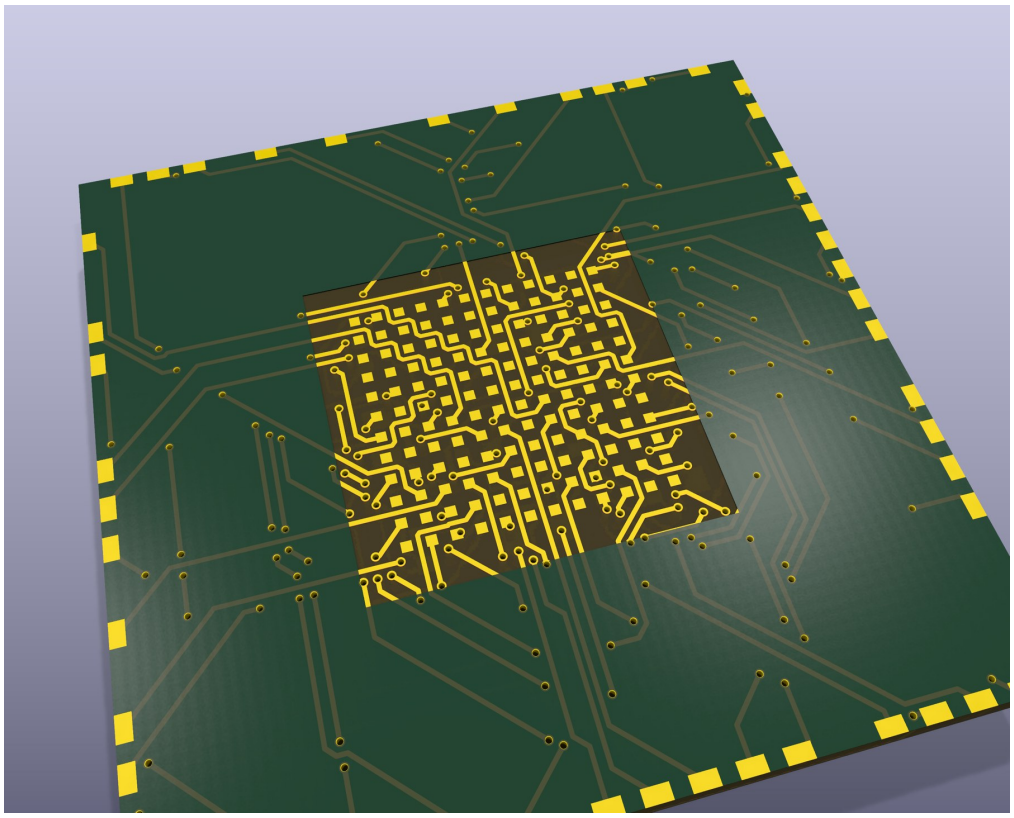
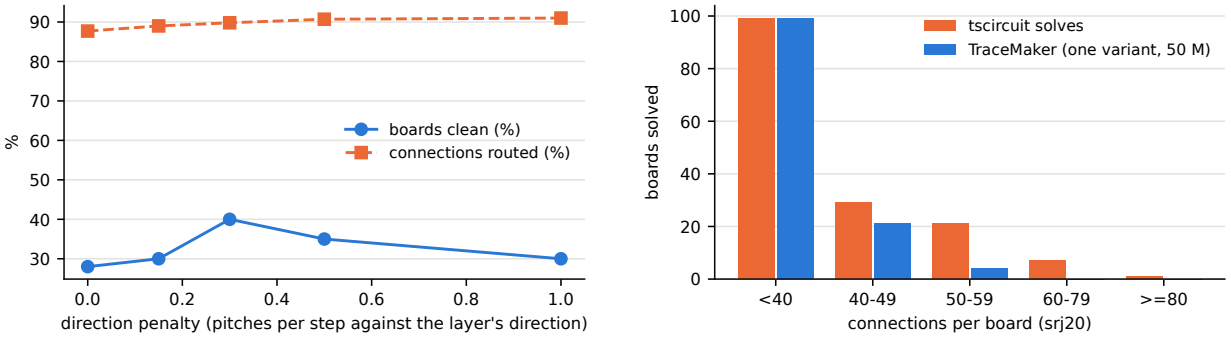


Figure 17: `srj19` sample 196 after routing, rendered by KiCad's 3D viewer (copper only).

- **A corrected fan-out model.** The first import of the K230 cases routed none of 12. The importer had overridden the layer of each breakout point with the layer named for its bus, but several buses' breakout points share coordinates and differ only by layer, so the override stacked two or three terminals on one layer and made them unreachable. Dropping the override (the importer's other features stayed) took K230 to 12 of 12. Also needed: plane terminations as zones, pads of nets with no connection point as keep-outs, and a board-outline margin.



(a) Direction penalty, 57 `srj20` boards that one variant first missed (30 M work).

(b) Where `srj20` is lost: boards solved by connection count.

Figure 18: Left: 0.3 gives the most clean boards; higher penalties route more connections but leave fewer boards fully clean. Right: the first profile (50 M work) against tscircuit; the shortfall is the boards of 50–79 connections.

7.4 What did not work

Every item below was measured at equal work and kept as a hidden option with its result recorded (`docs/16`, sections 10–11; decision D95), so nobody repeats it.

- **tscircuit's own routes as path guides** (soft or hard, radii of 1.5–8 pitches) on 12 boards: 440–499 of 570 connections against 499 without. A connection guided along another router's off-lattice geometry

does not complete the board. **Layer assignment** by crossing count (two-layer max-cut) or chord angle: clean boards fell from 18 % to 11–14 %.

- **Different orders** (longest first, deep package pins first): 76.7 % of connections routed fell to 54.9 % and 68.6 %.
- **Global routing** (three modes): 69.2–76.5 % against 76.7 %. **Negotiating from the first pass:** 72.7 %.
- **Limiting how many connections one search may rip up** (1, 2, 3): clean boards fell from 28 % to 2, 5 and 7 %.
- **A growing crossing cost** (the PathFinder schedule), **history weights** 2–100: worse or no effect.
- **Weighted A*** (heuristic weight 1.5, 2.5): 4 % clean. Path quality matters here.
- **Cheaper vias** alone (3 down to 0.3 mm): clean boards 0 to 2 of 24; **finer lattices**: 20 µm was worse than 40 at equal work.
- **A polishing phase** that reserves part of the budget to rebuild the neighbourhoods of the last open connections: closed 0–1 of 3–6 and lost more in the shortened main phase. **24 seeded variants on one hard board** all stopped at 33–40 of 52 connections.
- **More work** helps only a little: 200 million units on the 33 remaining `srj20` gap boards closed 6.

7.5 Why `srj20` is not met

TraceMaker's failures on these boards are mostly negotiation churn, not proved infeasibility: 585 of about 1,170 failure reports on `srj20` read "ripped up by another connection and not routed again". The boards of 50–79 connections are where tscircuit's router solves 28 and TraceMaker about 8 (Figure 18b). Rendering both routers' answers on the same board shows what differs: tscircuit's solution of one 53-connection board has 188 vias, 173 of them outside the ball array, and long straight runs on both layers; TraceMaker's best uses fewer vias and long detours along the rim. The untried candidate is a planner that decides layers and via sites per connection before the search starts; the cheap stand-ins above did not replace it.

7.6 Other changes since release 0.8.0

Besides the BGA work, the current code contains changes that were made after the 6 October edition:

- **Simple Route JSON** support at the edge of the engine (`srj-import`, `srj-export`) and benchmark scripts for the tscircuit sets (decision D90).
- **Bus routing** (opt-in): the connections of a bus run in pin order, each searched inside a guide around its neighbour's lane, with skew tuning and a `tracemaker buses` measurement command (D91). Marked experimental.
- **After-route passes** (off by default): widening power tracks, stitching vias for plane return paths against an in-engine zone refill, and a track-style metric (D92). Zones are refilled in the engine with exact polygon operations that follow KiCad 10, so plane boards are judged without calling KiCad for the fills (D85).
- **Instrumentation:** per-phase work counters in the route report (D88).

Release 0.9.0 on the main branch also merges outside contributions (plane-aware routing of all-SMD boards, a macOS build, layer limits for tracks, and rule-reading fixes); its tier results are recorded in the development log as 100, 72.5, 60.0 and 54.5 % clean on tiers A to D. The branch described in this section was cut before that release and is not merged to main or published; the two have not been combined or re-measured together.

7.7 Reproducing the BGA results

```
python3 bench/run.py --set srj19          # also srj20, srj19-gap2, srj20-gap2 (gap sets), one variant
python3 bench/fanout31_run.py             # --cases 55-57 --variants 8 --route-args=--no-gpu
build/report-venv/bin/python report/make_m14_figures.py
```

The numbers are in `bench/results/srj19-p*, srj20-p*, fan31-*`; the status page at port 8767 shows them as they were during the work. Decisions D94–D98 are in `docs/12-decisions.md`.

8 Limitations and next steps

- **The last few connections at fine-pitch packages.** Most boards that are not clean miss a handful of connections where two or three nets keep displacing each other at one package, or where the vias a crossing needs do not fit under KiCad's default rules. Neither global routing nor re-ordering helped; a lattice of half the pitch helps a little (one more clean board in tier B) at four times the work.
- **`srj20` is 24 boards short of tscircuit** (133 against 157), as is the 50–79 connection range in general (Section 7). `fanout31` cases 55–57 (T113-S3) and 61–69 (AM3352) are unsolved.
- **Large boards are time-limited.** On the biggest boards the router finishes only one or two negotiation passes in 120 s.
- **From-scratch placement is at 85 % clean, not the 90 % aimed for,** and a part that cannot be placed legally is set down beside the board with a message.
- **Dense packages:** 41 % of the 17 dense-package boards are clean (Freerouting 2.5: 24 %).
- **Edge-connector fingers under solder-mask openings** are left unrouted rather than routed with mask bridges. Freerouting's published "clean" counts only clearance violations and unrouted connections, so on these boards TraceMaker is judged more strictly.
- **More vias than Freerouting.** Even after clean-up TraceMaker uses about 1.4 times Freerouting 2.5's vias and 2.2 times Freerouting 1.9's (median, held-out boards), and it narrows some tracks below the design width.
- **The tier statistics use Freerouting's published numbers,** which allowed up to 30 minutes per board and are judged by Freerouting's own DRC. The held-out quality benchmark (Section 5.6) runs both Freerouting versions locally and judges them with KiCad, but covers 60 boards and was not repeated for this edition.
- **One width per net.** Net classes from impedance rules use the front-layer width on every layer.
- **Copper text** is modelled by generous estimated boxes rather than exact glyph outlines.
- **Not measured:** the placer against the SA-PCB annealer (that baseline did not build on the test machine), and the viewer's frame rate.
- **Not repeated on the current code:** the tier benchmark, the held-out quality study and the placement study (Sections 5, 5.6, 6); their numbers are those of release 0.8.0.
- **Not built:** a planner that fixes layers and via sites before routing, a gridless routing arm, learned net ordering, a GPU global router, per-layer track widths, and a schematic reader of TraceMaker's own (netlists come from `kicad-cli`).

A Reproducing the results

```
cmake --preset release && cmake --build --preset release && ctest --preset release
scripts/fetch_fixtures.sh            # pinned fixtures and derived sets
python3 bench/run.py --tier B --limit 40 --time 120 --jobs 2 --threads 8 --name my-run
build/release/src/app/tracemaker route board.kicad_pcb -o routed.kicad_pcb --time 120 --view
build/release/src/place/tracemaker-place in.kicad_pcb -o out.kicad_pcb \
    --mode auto --route-check 3000000
build/report-venv/bin/python report/make_figures.py   # figures for this report
build/report-venv/bin/python report/make_m14_figures.py
python3 report/make_test_report.py                    # tables of roadmap_testing.tex
```